

L33: Verification Workflow

Simulation Without a Black Box

Outline

V&V Distinction

V&V Workflow Diagram

Verification Checklist

Trace Debugging

Extreme-Condition Tests

Unit Tests with pytest

Code Review for SimPy Models

Common Bugs

Key Takeaways

Verification vs. Validation

Verification

Did we build the model right?

Does the simulation code faithfully implement the conceptual model?

Tool: code review, unit tests, trace debugging, analytical benchmarks.

Validation

Did we build the right model?

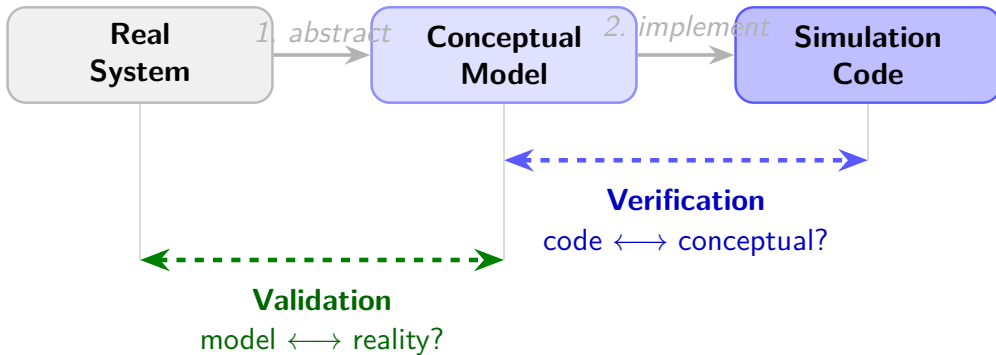
Does the conceptual model faithfully represent the real system?

Tool: historical data comparison, expert review, sensitivity checks.

Key Point

You cannot validate an unverified model — verification always comes first.

V&V Workflow



Verification must come first — you cannot validate an unverified model.

Verification Checklist

Code-level checks

- Event ordering: earlier events processed first
- Resource accounting: requests + releases balanced
- Statistics collected after warmup only
- Random seed is an explicit parameter
- No mutable global state

Behavioral checks

- Mass-balance: entities in = entities out + in-system
- Extreme conditions produce expected limits
- Single-entity trace matches hand calculation
- Results reproducible with same seed

Trace Debugging: Print State at Every Event

```
def customer(env, name, server, service_rate):
    arrive = env.now
    print(f"t={arrive:.2f}  {name} ARRIVES  queue={len(server.queue)}")
    with server.request() as req:
        yield req
        wait = env.now - arrive
        print(f"t={env.now:.2f}  {name} STARTS  wait={wait:.2f}")
        yield env.timeout(random.expovariate(service_rate))
        print(f"t={env.now:.2f}  {name} DEPARTS")
```

Rule

For every SimPy yield, print the simulation time, entity name, and key state variable *before and after* the yield.

Extreme-Condition Tests for M/M/1

Condition	Expected	Bug signal
$\lambda \rightarrow 0$	$\rho \rightarrow 0, W_q \rightarrow 0$	$\rho > 0$
$\lambda/\mu \rightarrow 1^-$	$W_q \rightarrow \infty$	finite W_q
$\rho > 1$	queue $\rightarrow \infty$	stable queue
$\mu \rightarrow \infty$	$W \rightarrow 1/\lambda$	$W \neq 1/\lambda$

Warning

$\rho > 1$ must be caught before running long experiments — an unstable simulation wastes compute and produces meaningless statistics.

Add a guard:

```
if lam / mu >= 1: raise  
ValueError("rho >= 1")
```

Unit Tests: Compare to Analytical Formula

```
import pytest
from simdes.models.queues import MM1Queue

def test_mm1_mean_wait():
    lam, mu = 0.8, 1.0          # rho = 0.8
    wq_theory = lam / (mu * (mu - lam))  # = 4.0
    q = MM1Queue(arrival_rate=lam, service_rate=mu, seed=42)
    result = q.run(sim_time=100_000)
    assert abs(result["Wq"] - wq_theory) / wq_theory < 0.05  # 5% tolerance

@pytest.mark.parametrize("lam", [0.1, 0.5, 0.9])
def test_mm1_utilisation(lam):
    q = MM1Queue(arrival_rate=lam, service_rate=1.0, seed=0)
    r = q.run(sim_time=50_000)
    assert abs(r["utilisation"] - lam) < 0.03
```


Code Review Checklist for SimPy Models

Structure

- Model is a class, not a script
- `run()` returns a dict of KPIs
- Seed passed to constructor
- No `import random` at module level

Logic

- `yield env.timeout()` never receives ≤ 0
- Resource capacity matches conceptual model

Statistics

- Warmup: statistics reset after warmup period
- Averages use time-weighted mean for stocks (L, Lq)
- Sample mean used for flows (W, Wq)

Common Bug

Forgetting to reset accumulators after warmup inflates means — the warm-up transient is the largest source of bias.

Common SimPy Bugs and Their Signatures

Bug	Symptom	Fix
Incorrect event ordering	Negative sojourn times	Check <code>env.now</code> ordering logic
Double-booking resource	Utilisation > 1	Ensure one <code>request()</code> per entity
Missing warmup removal	Biased mean estimates	Reset stats at warmup time
Wrong time-average formula	$L \neq \lambda W$	Use <code>env.now.deltas</code> for TWMA
Shared mutable list	Results vary with seed unpredictably	Pass seed; no module-level lists

Key Takeaways

L33 Summary

1. **Verification** = code matches conceptual model; **Validation** = model matches real system.
2. Use a systematic checklist: event ordering, resource balance, warmup, reproducibility.
3. **Trace debugging** — print every event — is the fastest way to locate logic errors.
4. Extreme-condition tests ($\lambda \rightarrow 0$, $\rho \rightarrow 1^-$, $\rho > 1$) are mandatory before any experiment.
5. Automate verification with pytest and compare to closed-form M/M/1 or M/M/c formulas.

Next: L34 — Extreme-condition and degenerate testing in depth.